

Access, Encapsulation, and Static Methods

The public and private keywords

In Java, the keywords `public` and `private` define the access of classes, instance variables, constructors, and methods.

`private` restricts access to only the class that declared the structure, while `public` allows for access from any class.

Encapsulation

Encapsulation is a technique used to keep implementation details hidden from other classes. Its aim is to create small bundles of logic.

The private Keyword

In Java, instance variables are encapsulated by using the `private` keyword. This prevents other classes from directly accessing these variables.

```
public class CheckingAccount{  
    // Three private instance variables  
    private String name;  
    private int balance;  
    private String id;  
}
```

Accessor Methods

In Java, accessor methods return the value of a `private` variable. This gives other classes access to that value stored in that variable without having direct access to the variable itself.

Accessor methods take no parameters and have a return type that matches the type of the variable they are accessing.

```
public class CheckingAccount{  
    private int balance;  
  
    //An accessor method  
    public int getBalance(){  
        return this.balance;  
    }  
}
```

Mutator Methods

In Java, mutator methods reset the value of a `private` variable. This gives other classes the ability to modify the value stored in that variable without having direct access to the variable itself.

Mutator methods take one parameter whose type matches the type of the variable it is modifying. Mutator methods usually don't return anything.

```
public class CheckingAccount{  
    private int balance;  
  
    //A mutator method  
    public void setBalance(int newBalance){  
        this.balance = newBalance;  
    }  
}
```

Local Variables

In Java, local variables can only be used within the scope that they were defined in. This scope is often defined by a set of curly brackets. Variables can't be used outside of those brackets.

```
public void exampleMethod(int  
exampleVariable){  
    // exampleVariable can only be used  
    inside these curly brackets.  
}
```

The this Keyword with Variables

In Java, the `this` keyword can be used to designate the difference between instance variables and local variables. Variables with `this.` reference an instance variable.

```
public class Dog{  
    public String name;  
  
    public void speak(String name){  
        // Prints the instance variable named  
name  
        System.out.println(this.name);  
  
        // Prints the local variable named  
name  
        System.out.println(name);  
    }  
}
```

The this Keyword with Methods

In Java, the `this` keyword can be used to call methods when writing classes.

```
public class ExampleClass{  
    public void exampleMethodOne(){  
        System.out.println("Hello");  
    }  
  
    public void exampleMethodTwo(){  
        //Calling a method using this.  
        this.exampleMethodOne();  
        System.out.println("There");  
    }  
}
```

